Solutions for CS218 Tutorial 1

This tutorial comprises all unsolved problems originally outlined in the course lecture materials.

1 Binary Search and Variants

1.1 Median of Two Sorted Arrays

Problem: Given two sorted arrays of size n, find the median of the union of the two arrays. Can you do this in $O(\log n)$ time?

Solution: To find the median of the two arrays, we need to find the average of the middle two elements of the merged arrays.

We partition both arrays such that exactly n elements lie in the left half of the merged array. Suppose we take i elements from A; then we must take $j = n - i$ elements from B. Thus, the left half consists of $A[0 \dots i-1] \cup B[0 \dots j-1]$ and the right half consists of the remaining elements.

For the partition to be valid, all elements in $A[0 \dots i-1]$ should be less than or equal to $B[j \dots n-1]$, equivalently $A[i-1] \leq B[j]$. Similarly, all elements in $B[0 \dots j-1]$ should be less than or equal to $A[i \dots n-1]$, or $B[j-1] \leq A[i]$. For simplicity, we can define $L_A = A[i-1]$, $L_B = B[j-1]$, $R_A = A[i]$ and $L_B = B[j]$.

We binary-search on $i \in [0, n]$:

- If $L_A > R_B$, we have taken too many elements from A, decrease i.
- If $L_B > R_A$, we have taken too few elements from A, increase i.

Since these conditions are monotone in i, binary search finds a valid partition in $O(\log n)$ time.

Once a valid partition is found, the median is simply $\frac{\max(L_A, L_B) + \min(R_A, R_B)}{2}$

1.2 Convex Function Minimization

Problem: Given a convex function $f(x)$ oracle, find an integer x which minimizes $f(x)$.

Solution: Assume f has a single minima in the interval $[l, r]$. At each iteration choose $m_1 = l + \frac{r-l}{3}$ and $m_2 = r - \frac{r-l}{3}$. If $f(m_1) < f(m_2)$ then the minimum cannot lie in $(m_2, r]$, so set $r \leftarrow m_2$; if $f(m_1) > f(m_2)$ then the minimum cannot lie in $[l, m_1)$, so set $l \leftarrow m_1$; if $f(m_1) = f(m_2)$ restrict to $[m_1, m_2]$. Repeat this process recursively upto a certain desired tolerance.

1.3 Land Redistribution

Problem: Given a list of landholdings $a_1, a_2, \dots, a_n$ and a floor value f, find the right ceiling value c.

Solution: Fix a feasible floor value f and define the land deficit $D = \sum_{i=1}^{n} \max(0,\, f - a_i)$ which is the total amount of land required to raise all holdings below f up to f. For any ceiling value c, define the excess land obtained by capping large holdings as $E(c) = \sum_{i=1}^{n} \max(0,\, a_i - c)$. The function $E(c)$ is continuous and monotonically non-increasing in c. A value c is feasible if and only if $E(c) \geq D$, since in this case the land taken from holders above c suffices to meet the deficit of holders below f. Hence the set of feasible ceilings is an interval $[0, c^*]$ for some maximal value c^*. Binary search on c suffices to find the maximum value of c, such that the constraint is satisfied.

1.4 Square Root (Division Method)

Problem: We learned the binary search method for $\sqrt{a}$. The exercise is to find a "division-like" algorithm (digit-by-digit calculation) for finding the square root.

Solution: An outline of the algorithm along with a justification for the same can be found at Why is square root by long division found so?.

2 Interval Problems

2.1 Total Length of Union

Problem: Given n intervals, find the total length of their union.

Solution: We want to compute the total length covered by a collection of n intervals. First, sort all intervals by their left endpoints. Let the sorted intervals be $(l_1, r_1), (l_2, r_2), \ldots, (l_n, r_n)$.

We scan the intervals from left to right while maintaining the current merged interval. Let $(\texttt{curL}, \texttt{curR})$ denote the union of all intervals processed so far that intersect the current position, and let `ans` be the total length accumulated.

Initially, set $\texttt{curL} = l_1$ and $\texttt{curR} = r_1$. For each $i = 2$ to n, compare (l_i, r_i) with $(\texttt{curL}, \texttt{curR})$:

- If $l_i > \texttt{curR}$, the intervals are disjoint. Then the current segment $[\texttt{curL}, \texttt{curR}]$ contributes $\texttt{curR} - \texttt{curL}$ to the union, so we add this to `ans` and start a new segment by setting $\texttt{curL} = l_i$ and $\texttt{curR} = r_i$.
- Otherwise, the intervals overlap, so they belong to the same merged segment. We extend the right endpoint by setting $\texttt{curR} = \max(\texttt{curR}, r_i)$.

After all intervals are processed, the final segment $[\texttt{curL}, \texttt{curR}]$ contributes $\texttt{curR} - \texttt{curL}$, which we add to `ans`. The resulting value `ans` is the total length of the union.

Sorting takes $O(n \log n)$ time and the linear scan takes $O(n)$ time, so the total running time is $O(n \log n)$.

2.2 Non-Overlapping Intervals Count

Problem: Given n intervals, find the number of intervals which do not overlap with any other interval.

Solution: First, sort the intervals by increasing left endpoint. Let the sorted intervals be $(l_1, r_1), (l_2, r_2), \ldots, (l_n, r_n)$.

We scan the intervals from left to right while maintaining the maximum right endpoint `maxR` of all previously seen intervals. An interval (l_i, r_i) overlaps with some earlier interval if $l_i \leq \texttt{maxR}$.

Similarly, to check whether (l_i, r_i) overlaps with a later interval, we observe that this happens exactly when $r_i \geq l_{i+1}$.

Thus, (l_i, r_i) does not overlap with any other interval if and only if

$$l_i > \texttt{maxR} \quad \text{and} \quad r_i < l_{i+1},$$

We update $\texttt{maxR} = \max(\texttt{maxR}, r_i)$ as we scan and count all intervals satisfying the above conditions.

The sorting step takes $O(n \log n)$ time and the scan takes $O(n)$ time, so the overall running time is $O(n \log n)$.

2.3 Maximum Intersection

Problem: Given n intervals, find the maximum number of mutually intersecting intervals (the point covered by the most intervals).

Solution: For each interval (l_i, r_i), create two events - a start event $(l_i, +1)$ and an end event $(r_i, -1)$. We sort all $2n$ events by their position; if two events have the same position, we process start events before end events.

We then sweep from left to right, maintaining a counter `cnt` equal to the number of intervals currently active (started but not yet ended). Initially `cnt` $= 0$. When we encounter a start event we increment `cnt`, and when we encounter an end event we decrement `cnt`. We keep track of the maximum value reached by `cnt` during the run; this value is the maximum number of intervals that cover a common point.

Sorting the $2n$ events takes $O(n \log n)$ time, and the sweep itself is linear, so the overall running time is $O(n \log n)$.

3 Algorithm Design and Analysis

3.1 Stock Market Profit

Problem: Given share prices for n days $p_1, p_2, \ldots, p_n$. You have to buy on one day and sell on a later day. Find the maximum profit possible in $O(n)$ time.

Solution: The idea is to traverse the prices once while keeping track of the lowest price observed so far and the best profit achievable.

Algorithm 1 Maximum Stock Profit

```
minPrice ← p_1
maxProfit ← 0
for i = 2 to n do
    maxProfit ← max(maxProfit, p_i − minPrice)
    minPrice ← min(minPrice, p_i)
end for
return maxProfit
```

Correctness We show that the algorithm returns the maximum possible profit. At the beginning of iteration i, minPrice stores the minimum value among $p_1, \ldots, p_{i-1}$. Thus, the quantity $p_i - \text{minPrice}$ is the maximum profit achievable by selling on day i. The algorithm updates maxProfit with the best such value over all days, ensuring that all valid buy–sell pairs are considered. Therefore, the algorithm returns the maximum possible profit.

3.2 The Celebrity Problem

Problem: In a party of n people, a celebrity is defined as someone who is known to everyone but knows no one. Can you find the celebrity in $O(n)$ queries?

Intuition. A celebrity is someone who is known by everyone but knows no one. Consider any two people a and b:

- If a knows b, then a cannot be a celebrity.
- If a does not know b, then b cannot be a celebrity.

Thus, with a single query `knows`(a, b), we can eliminate *at least one* person from being a celebrity.

By repeatedly comparing people pairwise, we can eliminate non-celebrities until only one possible candidate remains. If a celebrity exists, it must be this remaining person. Finally, we explicitly check whether this candidate satisfies the celebrity conditions.

Algorithm.

1. Initialize a candidate $c = 1$.
2. For $i = 2$ to n:

- If $\texttt{knows}(c, i) = 1$, then c cannot be a celebrity, so set $c \leftarrow i$.

3. Verify candidate c:
   - For all $i \neq c$, check:
   $$\texttt{knows}(c, i) = 0 \quad \text{and} \quad \texttt{knows}(i, c) = 1.$$
   - If all checks pass, c is the celebrity; otherwise, no celebrity exists.

Complexity. The elimination phase uses $n - 1$ queries. The verification phase uses at most $2(n - 1)$ queries. Hence, the total number of queries is $O(n)$.

4 Introductory Open Questions

4.1 Fibonacci Numbers

Problem: Can we compute the n^{th} Fibonacci number $F(n)$ in $O(\log n)$ arithmetic operations?

Solution 1: We all know that Fibonacci numbers are defined as

$$\begin{aligned} F(1) &= 1 \\ F(2) &= 1 \\ F(k) &= F(k-1) + F(k-2), \ \forall k \geq 3 \end{aligned}$$

Now, the most naive way to calculate $F(n)$ is to inductively calculate them and store them in an array of size n. This takes $O(n)$ steps. How do we go beyond this? The $\log n$ requirement reeks of trying to reduce our computations by a constant factor after each step so that we can reach the base case in logarithmic number of steps (This is quite similar, at least in spirit, to Binary Search as one wants to perform a structured search operation rather than iterating over everything in between). So, how do we do that?

Let's first try to see if there structure we can exploit.

$$\begin{aligned} F(n) &= F(n-1) + F(n-2) \\ F(n-1) + F(n-2) &= (F(n-2) + F(n-3)) + F(n-2) \\ &= 2F(n-2) + F(n-3) \\ 2F(n-2) + F(n-3) &= 2(F(n-3) + F(n-4)) + F(n-3) \\ &= 3F(n-3) + 2F(n-4) \\ &= 5F(n-4) + 3F(n-5) \\ &\vdots \end{aligned}$$

So, a pattern seems to emerge, the coefficients are of the form $(1, 1), (2, 1), (3, 2), (5, 3)$ which are exactly ... consecutive Fibonacci numbers. In fact, the above process of successive computation can be used as a proof by induction of the following fact (this is just a simple exercise - try it out if this is unclear):

$$F(n) = F(m+1)F(n-m) + F(m)F(n-m-1) \quad \forall m \geq 1, n \geq 3, m \leq n-2$$

The nature of this expression is a little difficult to understand in this form. However, considering two cases of the form n being even and odd leads to a simpler analysis. In that case, we can get the following equations:

- If $n = 2k, k \geq 2$, substituting $m = k$, we get:
$$\begin{aligned} F(2k) &= F(k+1)F(k) + F(k)F(k-1) \\ &= F(k)^2 + 2F(k)F(k-1) \end{aligned}$$
- If $n = 2k + 1, k \geq 1$, substituting $m = k$, we get:
$$\begin{aligned} F(2k+1) &= F(k+1)^2 + F(k)^2 \\ &= 2F(k)^2 + F(k-1)^2 + 2F(k)F(k-1) \end{aligned}$$

- $F(1) = 1$ and $F(2) = 1$

Using the first equations iteratively, we can always calculate $F(n)$ in $\theta(\log n)$ steps (The last two equations are just for handling base cases).

Solution 2: The Fibonacci numbers satisfy the linear recurrence

$$F(n) = F(n-1) + F(n-2).$$

Computing them directly takes $O(n)$ steps. The key idea is to represent this recurrence as a matrix transformation and then use fast exponentiation. Since matrix exponentiation can be done by repeated squaring, the number of arithmetic operations reduces to $O(\log n)$.

Define

$$\begin{pmatrix} F(n) \\ F(n-1) \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F(n-1) \\ F(n-2) \end{pmatrix}.$$

Thus,

$$\begin{pmatrix} F(n) \\ F(n-1) \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-1} \begin{pmatrix} F(1) \\ F(0) \end{pmatrix}.$$

1. Let
$$M = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}.$$
2. Compute M^{n-1} using exponentiation by squaring.
3. Multiply the result with $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$ to obtain $F(n)$.

Matrix exponentiation by squaring takes $O(\log n)$ matrix multiplications. Since each multiplication of 2×2 matrices uses constant arithmetic operations, $F(n)$ is computed in $O(\log n)$ arithmetic operations. Also, the formulas used in previous solution can be easily derived using Matrices (Give it a try).

4.2 Integer Multiplication

Problem: Multiplying two n-bit numbers using the standard method takes $O(n^2)$ time. Is $O(n^2)$ time necessary?

Solution: Let's take a look at the traditional bit multiplication and try to decipher if we can actually break the $O(n^2)$ barrier. Given two n bit integers $m_{n-1}m_{n-2}\ldots m_1m_0$ and $p_{n-1}p_{n-2}\ldots p_1p_0$, how do we multiply them:

$$\begin{array}{rrcrrr}
 & & & m_{n-1} & m_{n-2} \quad \ldots & m_1 \quad m_0 \\
 & & & p_{n-1} & p_{n-2} \quad \ldots & p_1 \quad p_0 \\
\hline
 & & & [m_{n-1}p_0] & [m_{n-2}p_0] \quad \ldots & [m_1p_0] \quad [m_0p_0] \\
 & & [m_{n-1}p_1] & [m_{n-2}p_1] & \ldots \quad [m_1p_1] & [m_0p_1] \quad \times \\
 & [m_{n-1}p_2] & [m_{n-2}p_2] & \ldots & [m_1p_2] \quad [m_0p_2] & \times \quad \times \\
 & & & & & \vdots
\end{array}$$

So, the problem here is considering every bit as separate. If we keep doing that, we will always end up with $\theta(n^2)$ terms to calculate. So, how do we circumvent this? The motivation comes from sorting. Instead of trying to deal with bits, we deal with blocks of bits (in fact, this is a very general algorithm design technique known as Divide-and-Conquer and will be extensively discussed later in the course).